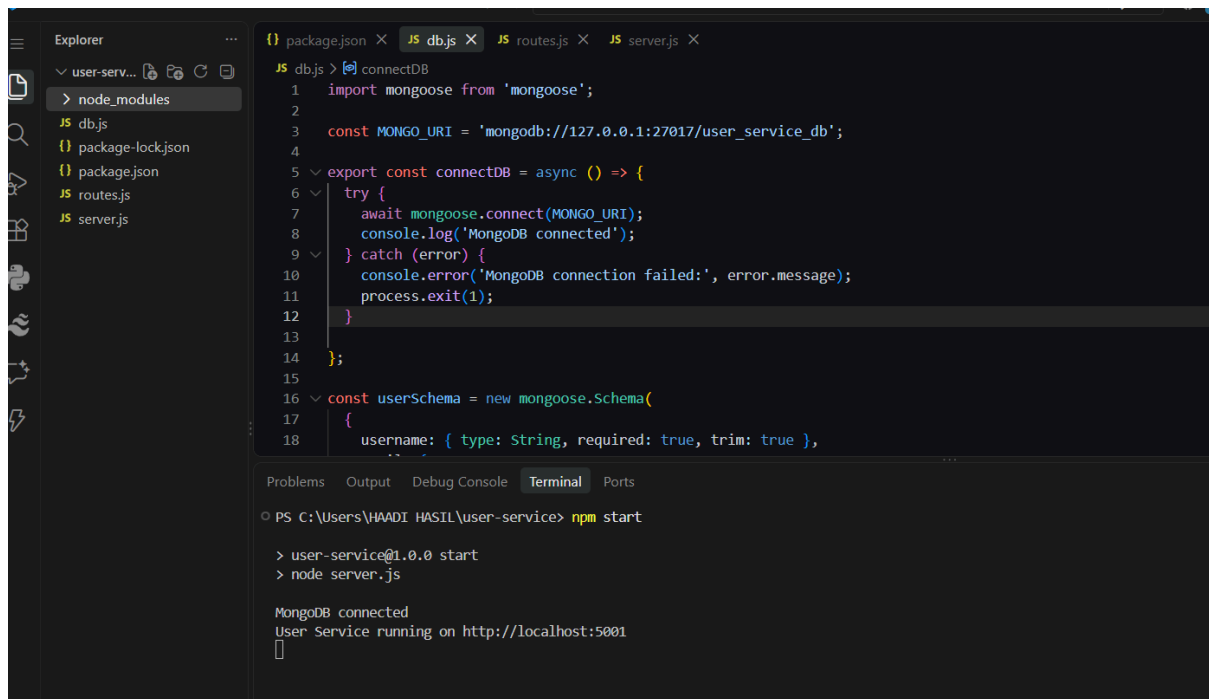


1.



The screenshot shows a Visual Studio Code editor with a project named 'user-service'. The Explorer sidebar on the left shows the file structure: 'node_modules', 'db.js', 'package-lock.json', 'package.json', 'routes.js', and 'server.js'. The main editor window has four tabs: 'package.json', 'db.js', 'routes.js', and 'server.js'. The 'db.js' tab is active, showing the following code:

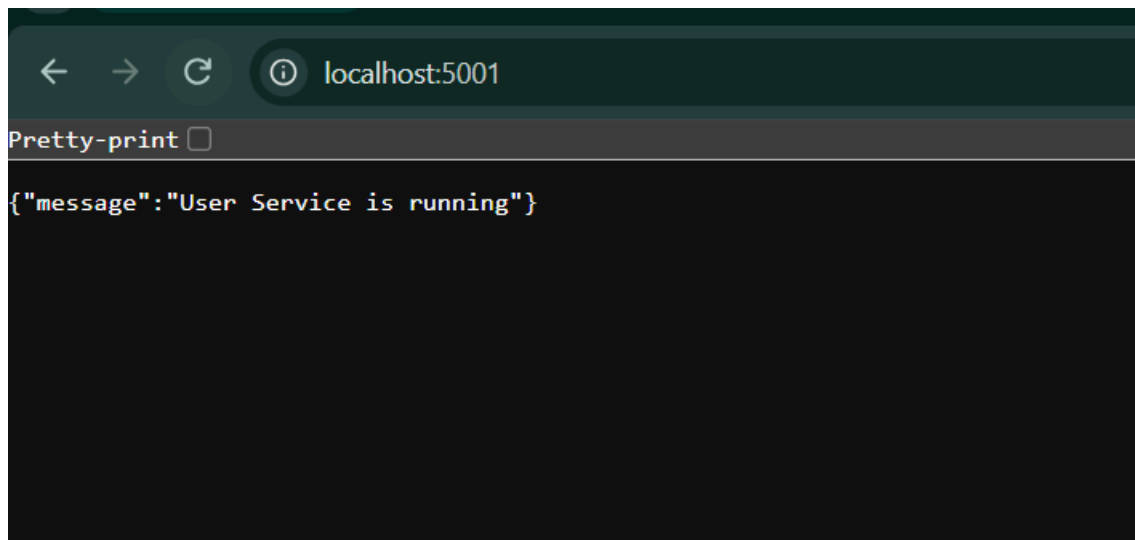
```
1  import mongoose from 'mongoose';
2
3  const MONGO_URI = 'mongodb://127.0.0.1:27017/user_service_db';
4
5  export const connectDB = async () => {
6    try {
7      await mongoose.connect(MONGO_URI);
8      console.log('MongoDB connected');
9    } catch (error) {
10     console.error('MongoDB connection failed:', error.message);
11     process.exit(1);
12   }
13 }
14
15
16 const userSchema = new mongoose.Schema(
17   {
18     username: { type: String, required: true, trim: true },
```

Below the code editor, the 'Terminal' tab is active, showing the command prompt output:

```
PS C:\Users\HAADI HASIL\user-service> npm start
> user-service@1.0.0 start
> node server.js

MongoDB connected
User Service running on http://localhost:5001
[]
```

2.



The screenshot shows a web browser window with the address bar set to 'localhost:5001'. Below the address bar, there is a 'Pretty-print' checkbox which is currently unchecked. The main content area of the browser displays the following JSON response:

```
{"message": "User Service is running"}
```

3.

The screenshot shows a REST client interface with a sidebar on the left containing 'COLLECTIONS' (My Collection, GET Get data, POST Post data) and 'ENVIRONMENTS' (SPECS, MOCKS, DATASETS, FLOWS, DOCUMENTS). The main area displays a POST request to 'http://localhost:5001/users'. The request body is a JSON object: { "username": "ravi", "email": "ravi@example.com", "profile": { "bio": "Computer Science student", "department": "CSE", "interests": ["Python", "MongoDB"], "social": { "linkedin": "ravi-linkedin" } } }. The response status is '201 Created' with a body of 542 B. The response body is also shown as a JSON object: { "username": "ravi", "email": "ravi@example.com", "profile": { "bio": "Computer Science student", "department": "CSE", "interests": ["Python", "MongoDB"], "social": { "linkedin": "ravi-linkedin" } } }.

```
1 {
2   "username": "ravi",
3   "email": "ravi@example.com",
4   "profile": {
5     "bio": "Computer Science student",
6     "department": "CSE",
7     "interests": [
8       "Python",
9       "MongoDB"
10    ],
11    "social": {
12      "linkedin": "ravi-linkedin"
13    }
14  }
15 }
```

```
1 {
2   "username": "ravi",
3   "email": "ravi@example.com",
4   "profile": {
5     "bio": "Computer Science student",
6     "department": "CSE",
7     "interests": [
8       "Python",
9       "MongoDB"
10    ],
11    "social": {
12      "linkedin": "ravi-linkedin"
13    }
14  }
15 }
```

4.

POST http: • GET http: • POST http: • POST http: • POST http: + ▾ No environment ▾

 <http://localhost:5001/users/6aae06089569a2234e...>  Save ▾  Share 

GET ▾ <http://localhost:5001/users/6aae06089569a2234ecb3f1e> Send ▾

≡ Docs **Params** Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body ▾  200 OK • 48 ms • 537 B •  | ...

{ } JSON ▾  Preview  Visualize ▾  | ≡ | 🔍 |  | 

```
1  {
2      "_id": "6aae06089569a2234ecb3f1e",
3      "username": "ravi",
4      "email": "ravi@example.com",
5      "profile": {
6          "bio": "Computer Science student",
7          "department": "CSE",
8          "interests": [
9              "Python",
10             "MongoDB"
11          ],
12          "social": {
13              "linkedin": "ravi-linkedin"
14          }
15      },
16      "createdAt": "2026-09-19T03:48:24.594Z",
17      "updatedAt": "2026-09-19T03:48:24.594Z",
18      "__v": 0
19  }
```

5.

5.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:5001/activities`
- Method:** `POST`
- Body (raw):**

```
1 {
2   "user_id": "6aae06089569a2234ecb3f1e",
3   "action": "VIEW_BOOK"
4 }
```
- Response:** `201 Created` (13 ms, 380 B)
- Response Body (JSON):**

```
1 {
2   "user_id": "6aae06089569a2234ecb3f1e",
3   "action": "VIEW_BOOK",
4   "_id": "6aae07d29569a2234ecb3f23",
5   "timestamp": "2026-09-19T03:56:02.666Z",
6   "__v": 0
7 }
```

6.

http://localhost:5001/activities/6aae06089569a2234ecb3f1e

GET http://localhost:5001/activities/6aae06089569a2234ecb3f1e

Params Authorization Headers 0 Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK • 57 ms • 1.33 KB

JSON Preview Visualize

```
39  {
40    "user_id": "6aae06089569a2234ecb3f1e",
41    "action": "LOGIN",
42    "timestamp": "2026-09-19T03:52:07.163Z",
43    "__v": 0
44  },
45  {
46    "_id": "6aae06d89569a2234ecb3f20",
47    "user_id": "6aae06089569a2234ecb3f1e",
48    "action": "SEARCH_BOOK",
49    "timestamp": "2026-09-19T03:51:52.351Z",
50    "__v": 0
51  },
52  {
53    "_id": "6aae06989569a2234ecb3f1f",
54    "user_id": "6aae06089569a2234ecb3f1e",
55    "action": "VIEW_BOOK",
56    "timestamp": "2026-09-19T03:50:48.154Z",
57    "__v": 0
58  }
59 }
```

7.

The screenshot shows a REST client interface with the URL `http://localhost:5001/users` and a `POST` method. The request body is a JSON object: `{ "username": "TestUser" }`. The response status is `400 Bad Request` with a response time of `6 ms` and a size of `347 B`. The response body is a JSON object: `{ "message": "Failed to create user", "error": "User validation failed: email: Path 'email' is required." }`.

```
1 {
2   "username": "TestUser"
3 }
```

Body Cookies Headers 7 Test Results 400 Bad Request 6 ms 347 B

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Failed to create user",
3   "error": "User validation failed: email: Path 'email' is required."
4 }
```

The screenshot shows a REST client interface with the URL `http://localhost:5001/users` and a `POST` method. The request body is a JSON object: `{ "username": "TestUser" }`. The response status is `400 Bad Request` with a response time of `6 ms` and a size of `347 B`. The response body is a JSON object: `{ "message": "Failed to create user", "error": "User validation failed: email: Path 'email' is required." }`.

```
1 {
2   "username": "TestUser"
3 }
```

Body Cookies Headers 7 Test Results 400 Bad Request 6 ms 347 B

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Failed to create user",
3   "error": "User validation failed: email: Path 'email' is required."
4 }
```